

A quick introduction to FLR

20 February, 2026

The Fisheries Library in R (**FLR**) is a collection of tools for quantitative fisheries science, developed in the R language, that facilitates the construction of bio-economic simulation models of fisheries systems as well as the application of a wide range of quantitative analysis.

FLR builds on the powerful R environment and syntax to create a domain-specific language for the quantitative analysis of the expected risks and impacts of fisheries management decisions. The classes and methods in FLR consider uncertainty an integral part of our knowledge of fisheries systems.

Required packages

To follow this tutorial you should have installed the following packages:

- CRAN: latticeExtra, gridExtra, ggplot2
- FLR: FLCore, FLa4a, ggplotFL, FLBRP, FLasher

You can do so as follows:

```
install.packages(c("latticeExtra", "gridExtra", "ggplot2"))
install.packages(c("FLCore", "ggplotFL", "FLa4a", "FLBRP", "FLasher"),
  repos=c(FLR="https://flr.r-universe.dev", CRAN="https://cloud.r-project.org"))
```

Classes and methods

The *FLCore* package defines a series of data structures that represent different elements of the fishery system. These structures are defined using R's S4 class system, while functions to operate on those classes are defined as methods. In this way function call (e.g. `plot()`) can be used on different classes and their behaviour is consistent with the class structure and content. For further information on the S4 class system, please look at the introductory text on the **methods** package.

FLQuant

The *FLQuant* class is the basic data structure for **FLR**, and it is in essence a 6D array on which data and outputs can be stored, structured along dimensions related to age or length, year, unit (e.g. sex), season, area and interactions. It is also able to keep track of the units of measurement employed, as a simple character vector, although various operations on these objects are aware of the units of measurements contained in them.

We will be creating and altering *FLQuant* objects as we go along, but we can look briefly at some basic operations, as the syntax will then be used in other classes in **FLR**.

Let's first load the necessary packages in our session. The **FLCore** package

```
library(FLCore)
library(ggplotFL)
```

As it is the case for R classes, for example *data.frame*, classes in **FLR** can be constructed using a constructor method with the same name as the class, in this case `FLQuant()`.

```
FLQuant(1:10)
```

An object of class "FLQuant"

```
, , unit = unique, season = all, area = unique
```

```
      year
quant 1  2  3  4  5  6  7  8  9 10
all   1  2  3  4  5  6  7  8  9 10
```

```
units: NA
```

The *FLQuant* constructor can take a variety of inputs, like a vector, a matrix or an array, and also allows you to specify a number of arguments. We will only look now at three of them, `units` for the units of measurement, `quant` for the name of the first dimension, and `dimnames` for the dimension names of the object. For example, to construct an object holding (random) data in tonnes on catches for ages 1 to 4, over a period of 6 years, one could call

```
flq <- FLQuant(rlnorm(60), dimnames=list(age=1:4, year=2012:2017), units="t")
```

```
flq
```

An object of class "FLQuant"

```
, , unit = unique, season = all, area = unique
```

```
      year
age 2012 2013 2014 2015 2016 2017
  1 0.604 0.974 0.991 1.685 1.364 5.057
  2 1.938 2.099 3.139 0.850 0.534 0.700
  3 0.248 1.517 5.280 0.612 0.595 1.195
  4 3.147 6.164 1.062 0.936 1.432 0.815
```

```
units: t
```

The object has six dimensions, although some of them (`unit`, `season`, `area` and `iter`) are not used in this case. We can inspect the object in various ways

```
# A summary of structure and data
```

```
summary(flq)
```

An object of class "FLQuant" with:

```
dim:   age year unit season area iter
      4   6   1    1      1    1
units: t
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	%NAs
0.25	0.79	1.13	1.79	1.98	6.16	0.00

```
# dimnames
```

```
dimnames(flq)
```

```
$age
```

```
[1] "1" "2" "3" "4"
```

```
$year
```

```
[1] "2012" "2013" "2014" "2015" "2016" "2017"
```

```
$unit  
[1] "unique"
```

```
$season  
[1] "all"
```

```
$area  
[1] "unique"
```

```
$iter  
[1] "1"
```

```
# dims  
dim(flq)
```

```
[1] 4 6 1 1 1 1
```

```
# units  
units(flq)
```

```
[1] "t"
```

It can be also subset and modified. Note that contrary to arrays in R one doesn't need to specify all dimension within square brackets, the **FLR** method understands there are more dimensions after the last one the user wants to use

```
# Extract (by location) the first year  
flq[, 1]
```

An object of class "FLQuant"
, , unit = unique, season = all, area = unique

```
      year  
age 2012  
  1 0.604  
  2 1.938  
  3 0.248  
  4 3.147
```

```
units: t
```

```
# in a R array, to get the same results, we'd need to do  
flq[, 1,,,]
```

An object of class "FLQuant"
, , unit = unique, season = all, area = unique

```
      year  
age 2012  
  1 0.604  
  2 1.938  
  3 0.248  
  4 3.147
```

```
units: t
```

```
# Extract (by name) year 2013  
flq[, "2013"]
```

An object of class "FLQuant"
, , unit = unique, season = all, area = unique

```
year
age 2013
  1 0.974
  2 2.099
  3 1.517
  4 6.164
```

units: t

```
# Set catches on age 1 to zero
flq[1] <- 0
flq
```

An object of class "FLQuant"
, , unit = unique, season = all, area = unique

```
year
age 2012 2013 2014 2015 2016 2017
  1 0.000 0.000 0.000 0.000 0.000 0.000
  2 1.938 2.099 3.139 0.850 0.534 0.700
  3 0.248 1.517 5.280 0.612 0.595 1.195
  4 3.147 6.164 1.062 0.936 1.432 0.815
```

units: t

as it can be used in many operations

```
# Product with scalar
flq * 10
```

An object of class "FLQuant"
, , unit = unique, season = all, area = unique

```
year
age 2012 2013 2014 2015 2016 2017
  1  0.00  0.00  0.00  0.00  0.00  0.00
  2 19.38 20.99 31.39  8.50  5.34  7.00
  3  2.48 15.17 52.80  6.12  5.95 11.95
  4 31.47 61.64 10.62  9.36 14.32  8.15
```

units: t

```
# Addition with another FLQuant of the same dimensions
flq + (flq * 0.20)
```

An object of class "FLQuant"
, , unit = unique, season = all, area = unique

```
year
age 2012 2013 2014 2015 2016 2017
  1 0.000 0.000 0.000 0.000 0.000 0.000
  2 2.326 2.519 3.767 1.020 0.641 0.840
  3 0.298 1.821 6.336 0.735 0.714 1.435
  4 3.777 7.396 1.275 1.123 1.718 0.978
```

```
units: t
# Sum along years
yearSums(flq)

An object of class "FLQuant"
, , unit = unique, season = all, area = unique

  year
age 1
  1  0.00
  2  9.26
  3  9.45
  4 13.56

units: t
```

More information

To learn more about the class, check the `FLQuant` help page.

Loading your data

The first step in an analysis is to get the relevant data into the system. We will need to construct objects of certain **FLR** classes, but first data is loaded into the R session using any of the tools available in the language: `read.csv` for CSV files, `readVPA` and others for fisheries legacy file formats, ...

In this example we will create an object of class *FLStock*, a representation of both inputs and outputs involved in fitting a stock assessment model, from a CSV file containing a table with five columns:

- *slot*, the name of the input quantity, in this case either landings at age, *landings.n*, weight-at-age in the landings, *landings.wt*, and the same quantities for discards, *discards.n* and *discards.wt*.
- *age* and *year*
- *data*, the numeric values themselves.
- *units*, a character string for the units of measurement employed: *kg* and *1000* in this case for kilograms and thousands.

This multi-column structure contains in effect a 2-D version of the set of 6-D arrays that form an *FLStock* object. We can observe this equivalence by converting our example object, `ple4`, into a single `data.frame`

```
data(ple4)
head(as.data.frame(ple4))
```

slot	age	year	unit	season	area	iter	data
catch	all	1957	unique	all	unique	1	78360.36
catch	all	1958	unique	all	unique	1	88785.20
catch	all	1959	unique	all	unique	1	105186.13
catch	all	1960	unique	all	unique	1	117974.54
catch	all	1961	unique	all	unique	1	119540.66
catch	all	1962	unique	all	unique	1	126290.43

```
tail(as.data.frame(ple4))
```

	slot	age	year	unit	season	area	iter	data
8169	m.spwn	5	2017	unique	all	unique	1	0
8170	m.spwn	6	2017	unique	all	unique	1	0
8171	m.spwn	7	2017	unique	all	unique	1	0
8172	m.spwn	8	2017	unique	all	unique	1	0
8173	m.spwn	9	2017	unique	all	unique	1	0
8174	m.spwn	10	2017	unique	all	unique	1	0

The file is downloaded into a temporary folder, and uncompressed. Simply change the value of `dir` to save the file in another folder.

```
dir <- tempdir()
download.file("http://www.flrproject.org/doc/src/ple4.csv.zip", file.path(dir, "ple4.csv.zip"))
unzip(file.path(dir, "ple4.csv.zip"), exdir=dir)
```

The CSV file can now be loaded as a `data.frame` using `read.csv` and inspected

```
dat <- read.csv(file.path(dir, "ple4.csv"))
head(dat)
```

slot	age	year	data	units
discards.n	1	1957	32356	1000
discards.n	2	1957	45596	1000
discards.n	3	1957	9220	1000
discards.n	4	1957	909	1000
discards.n	5	1957	961	1000
discards.n	6	1957	25	1000

This `data.frame` contains the time series of landings and discards at age, in thousands, and the corresponding mean weights-at-age, in kg, for North Sea plaice (*Pleuronectes platessa*, ICES ple.27.420).

We can create an object to store the landings-at-age data by subsetting it from the `data.frame` and passing all columns but 'slot'

```
landn <- subset(dat, slot=="landings.n", select=-slot)
```

and then convert into an *FLQuant* using

```
landsn <- as.FLQuant(landn)
```

The object can now be inspected and plotted using the FLR-defined methods

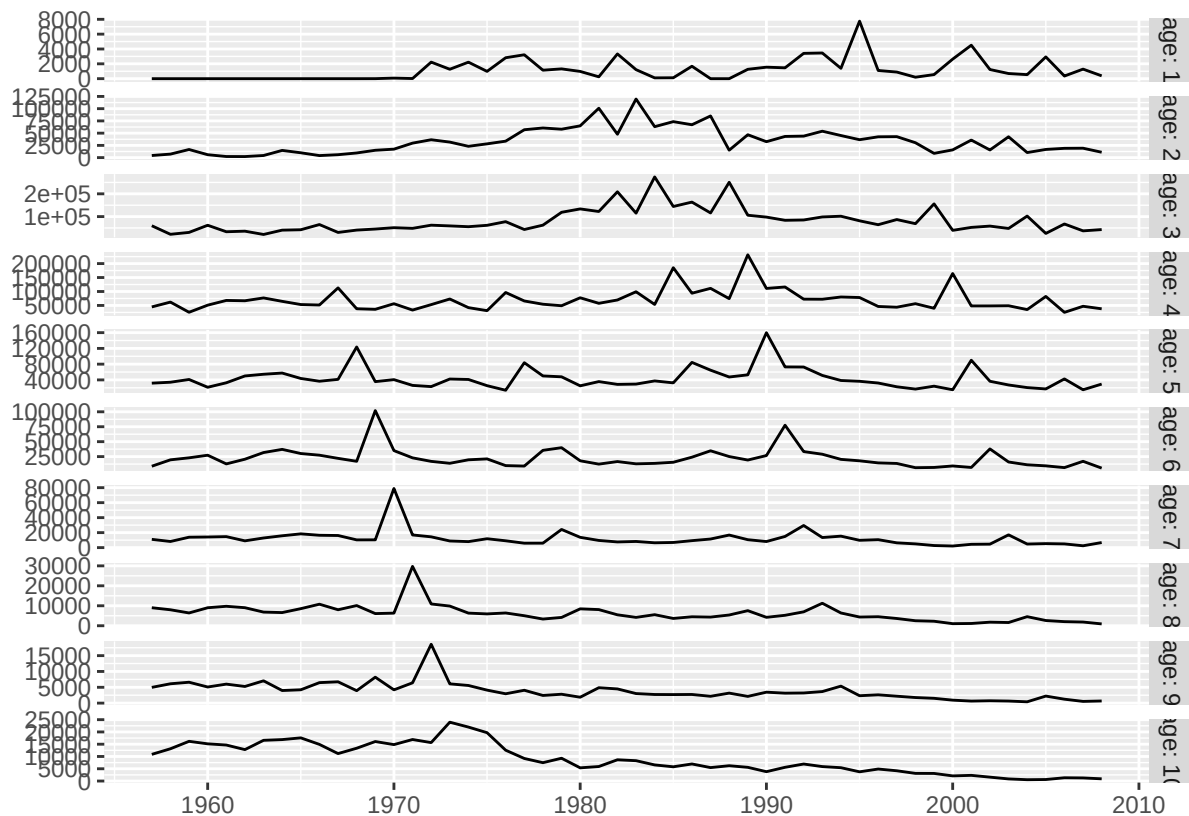
```
summary(landsn)
```

An object of class "FLQuant" with:

```
dim:   age year unit season area iter
      10  52   1    1      1    1
units: 1000
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    %NAs
      0     4548   14030   28141   40536   274209      0
```

```
plot(landsn)
```



In a similar way, we can now convert the input *data.frame*, containing data for four data elements, as specified in the *slots* column, into an *FLStock* object, `nple4`. The *FLStock* class has the following slots: *catch*, *catch.n*, *catch.wt*, *discards*, *discards.n*, *discards.wt*, *landings*, *landings.n*, *landings.wt*, *stock*, *stock.n*, *stock.wt*, *m*, *mat*, *harvest*, *harvest.spwn* and *m.spwn*. The names used in the *slot* column need to match the *FLStock* class for this method to work. See `?FLStock` for more information.

```
nple4 <- as.FLStock(dat)
```

```
summary(nple4)
```

An object of class "FLStock"

Name:

Description:

Quant: age

Dims:	age	year	unit	season	area	iter
	10	52	1	1	1	1

Range:	min	max	pgroup	minyear	maxyear	minfbar	maxfbar
	1	10	10	1957	2008	1	10

Metrics:

```
rec: NA - NA (NA)
ssb: 0 - 0 (NA)
catch: NA - NA (NA)
fbar: NA - NA (NA)
```

To complete this *FLStock* object we will need to specify the natural mortality, *m*, in this case as a constant value of 0.1 for all ages and years,

```
m(nple4) <- 0.1
```

the proportion of natural and fishing mortality that takes place before spawning, assumed to be zero in both cases,

```
m.spwn(nple4) <- harvest.spwn(nple4) <- 0
```

and the maturity at age, as the proportion mature, as a vector repeated for all years.

```
mat(nple4) <- c(0, 0.5, 0.5, rep(1, 7))
```

We now must compute the overall landings and discards, in biomass, from the age-disaggregated values,

```
landings(nple4) <- computeLandings(nple4)
discards(nple4) <- computeDiscards(nple4)
```

and then the catch slots from both landings and discards

```
catch(nple4) <- computeCatch(nple4, slot="all")
```

The mean weight-at-age in the stock needs to be provided. In this case we will assume it is the same as the one computed from the catch sampling programme

```
stock.wt(nple4) <- catch.wt(nple4)
```

We finalize by specifying the fully selected age range, used in the calculation of an overall index of fishing mortality, for example as the mean, using `fbar()`, or as the maximum value, using `fapex()` across those ages. This information is part of the object's *range*.

```
range(nple4, c("minfbar", "maxfbar")) <- c(2, 6)
```

If we now inspect the resulting object we can see that all calculated slots have been assigned the corresponding *units* of measurement, except for those that will hold the estimates coming from a stock assessment model: *stock*, *stock.n* and *stock.wt* for the estimates of abundance, and *harvest* for the estimates of fishing mortality at age.

```
summary(nple4)
```

An object of class "FLStock"

Name:

Description:

Quant: age

Dims:	age	year	unit	season	area	iter
	10	52	1	1	1	1

Range:	min	max	pgroup	minyear	maxyear	minfbar	maxfbar
	1	10	10	1957	2008	2	6

Metrics:

rec: NA - NA (NA)

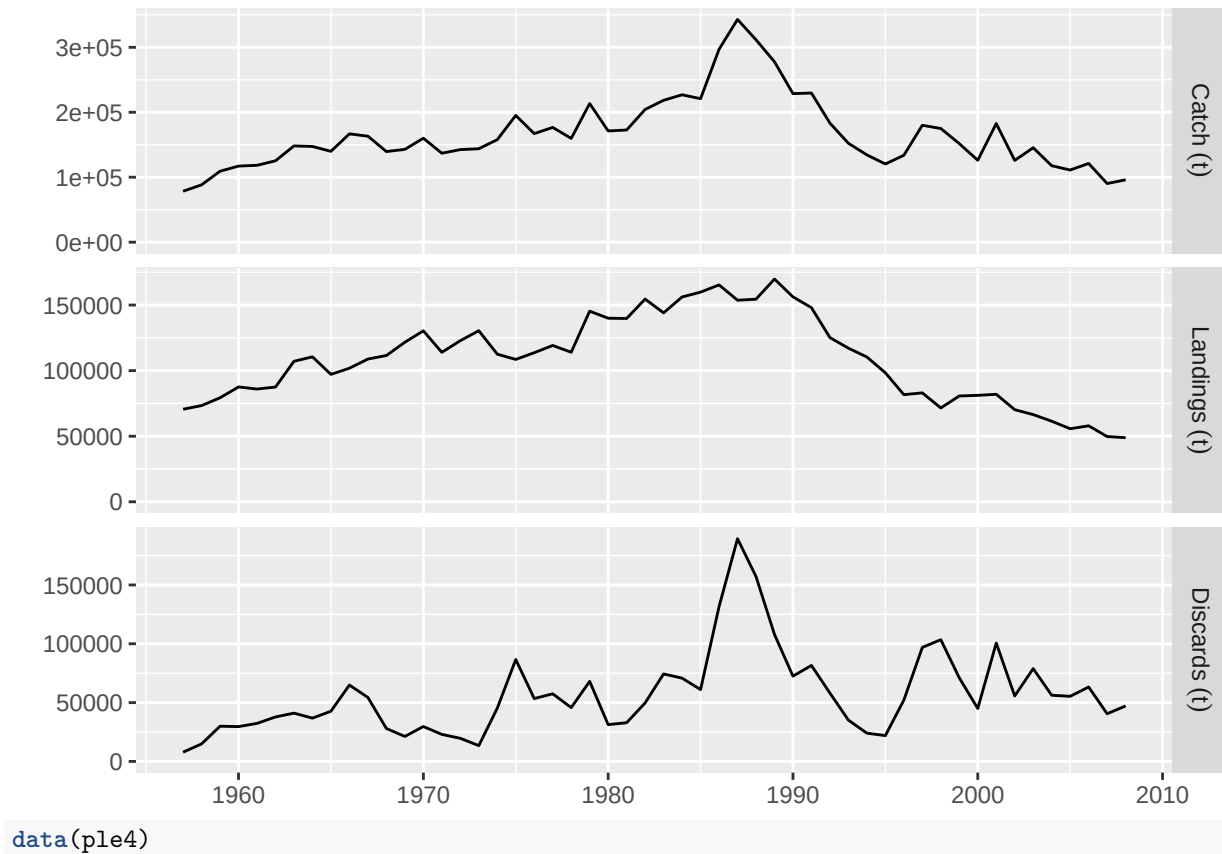
ssb: 0 - 0 (NA)

catch: 78423 - 342985 (t)

fbar: NA - NA (NA)

```
#plot(metrics(nple4, Catch=catch, Landings=landings))
```

```
plot(nple4)
```



More information

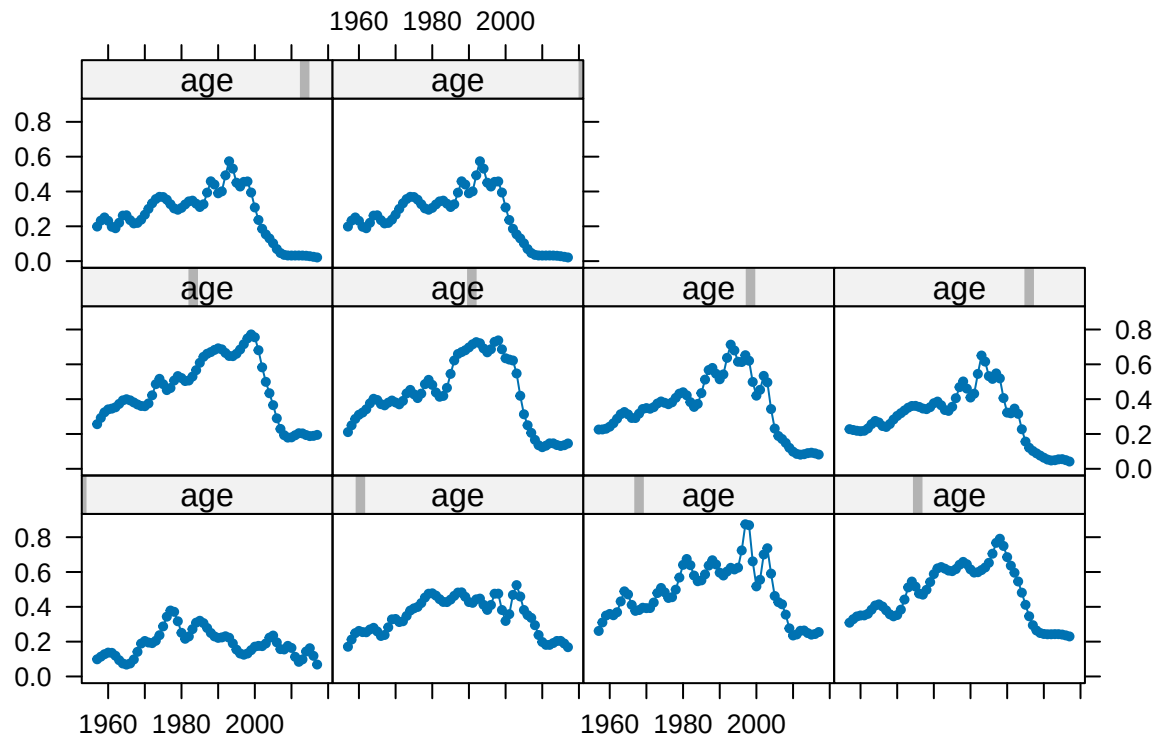
Please see the Loading your data into FLR tutorial for more examples on how to get your data into into **FLR**.

Visualizing and plotting

As already shown, **FLR** classes always have a `plot` method defined that provides a basic visual exploration of the object contents. But you are not limited to those plots, and other methods are available to allow you to build the plots you need.

The **FLCore** package provides a set of those plot methods based on R's lattice package, as well as versions of lattice's methods that work directly on **FLR** objects, for example `xyplot`

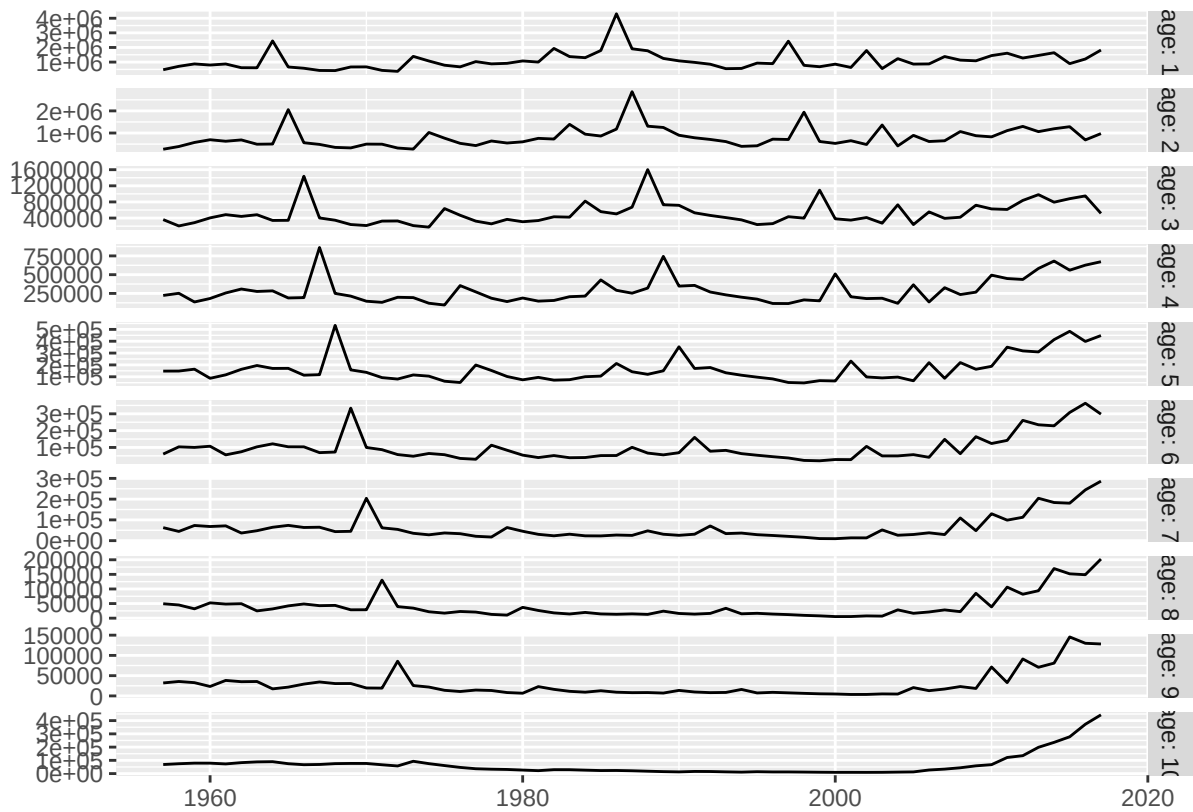
```
xyplot(data~year|age, harvest(ple4), xlab="", ylab="", type="b", cex=0.5, pch=19)
```



In alternative the **ggplotFL** package, which we have already loaded, can also be used, which provides a new set of plot methods based on **ggplot2**, from which all the plots shown so far in this tutorial have been generated.

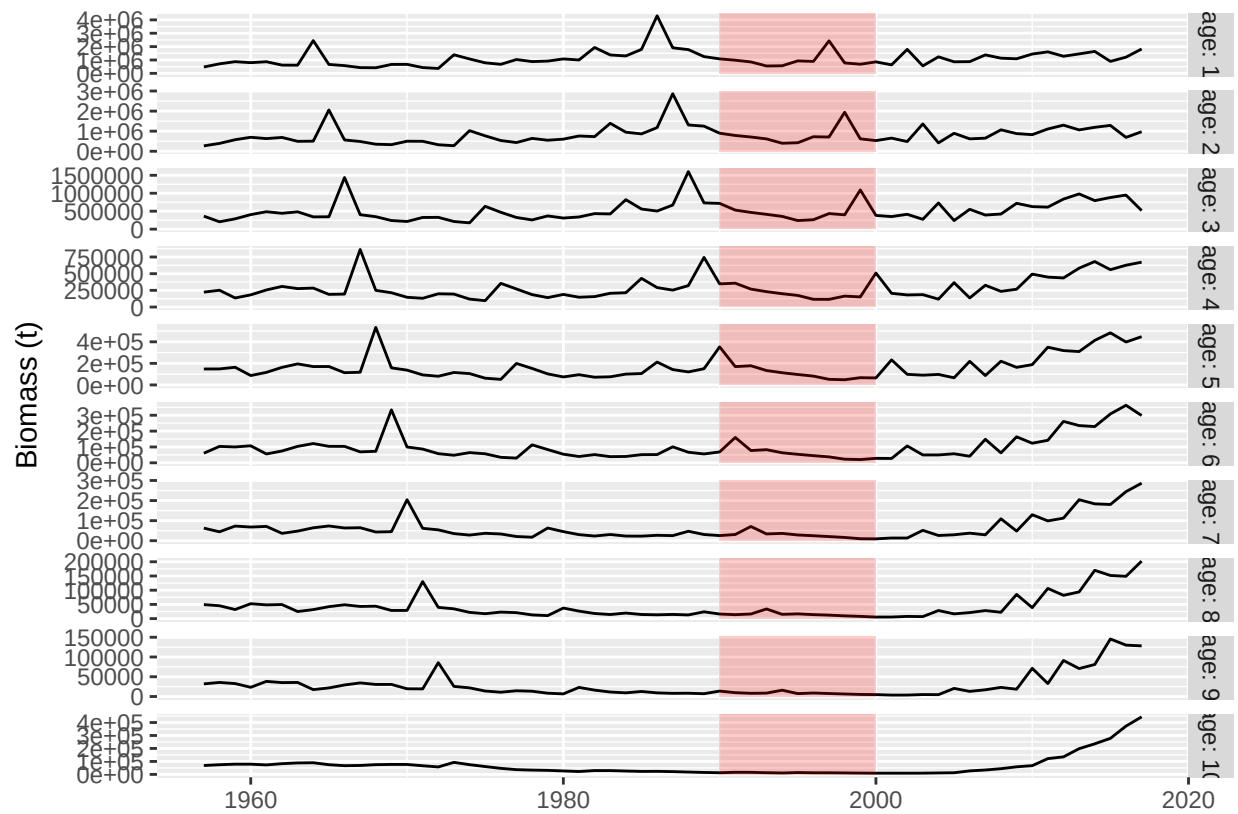
One important advantage of **ggplot2** is that plots are returned as objects that can be modified and extended. For example, the basic plot for the abundances at age *FLQuant* using **ggplotFL** would be this one

```
plot(stock.n(ple4))
```



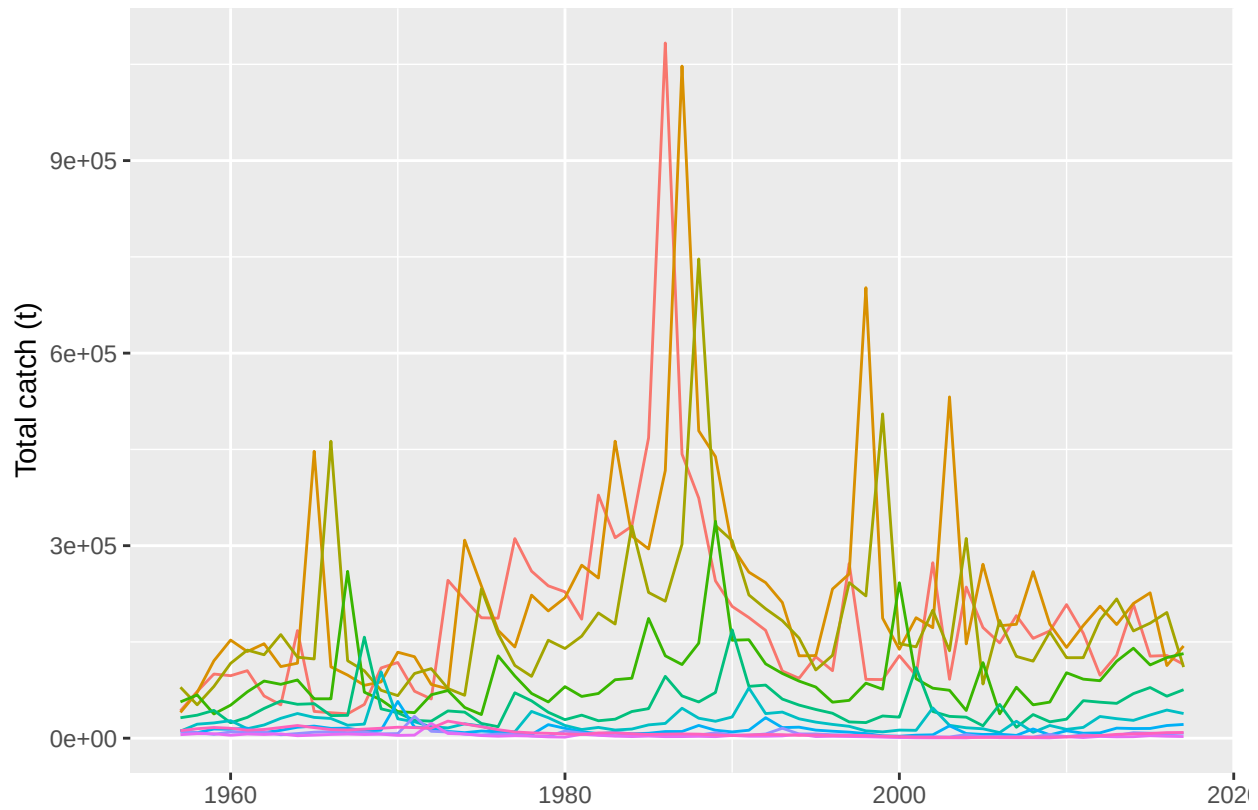
in which each age is shown in a different horizontal panel (*facet*). Now we can modify this plot in various ways by adding extra **ggplot2** commands to it, for example by adding a label to the y axis, shading a certain period of interest

```
plot(stock.n(ple4)) +
  # Add y label
  ylab("Biomass (t)") +
  # Draw rectangle between years 1990 and 2000
  annotate("rect", xmin = 1990, xmax = 2000, ymin = 0, ymax = Inf,
    # in semi-transparent red
    alpha = .2, fill='red')
```



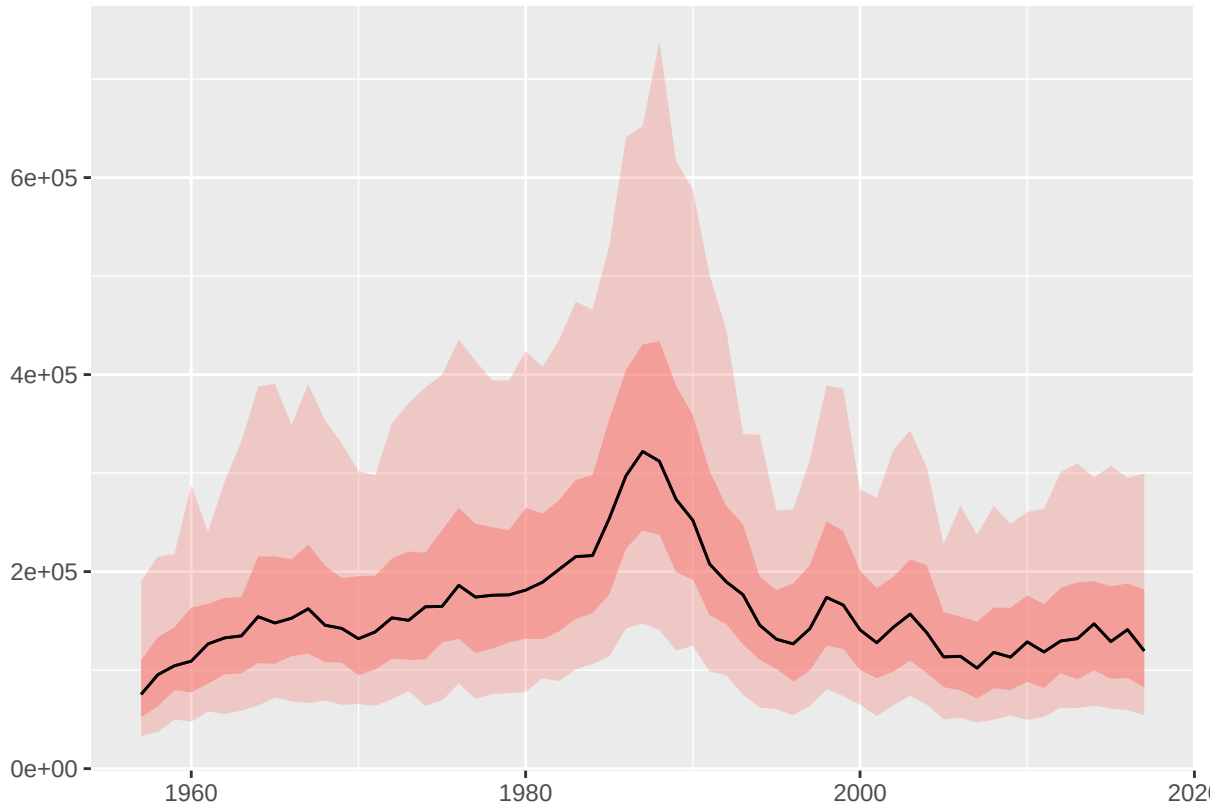
The basic method in **ggplot2** taking a data input, **ggplot**, has also been defined for some *FLCore* class, which simplifies constructing plots based on objects of those classes from scratch. For example, we can plot our catch-at-age matrix from *ple4* to show the signal of strong cohorts in the data using

```
ggplot(data=catch.n(ple4), aes(x=year, y=data, group=age)) +
  geom_line(aes(colour=as.factor(age))) +
  ylab("Total catch (t)") + xlab("") + theme(legend.position="none")
```



ggplotFL deals with multiple iterations in **FLR** objects by calculating quantiles along the *iter* dimension and plotting them as areas and lines. For example, an object with random lognormal noise around the catch time series would look like this

```
plot(rlnorm(250, log(catch(p1e4)), 0.5))
```



where the 50% and 95% intervals are shown as a red region and a dotted line respectively, while the median value is shown as a black line.

Note that plot methods on both **FLCore** and **ggplotFL** work by converting various **FLR** objects to a class that **lattice** and **ggplot2** can understand, *data.frames*.

Coercing a 6D array like an *FLQuant* to a 2D *data.frame* requires generating one column per dimension, named as those, plus an extra column for the values stored in the object. This column is named **data**, thus the use of **data** in various plot formulas above. For classes with multiple slots, like *FLStock*, an extra column is necessary, which will be called **slot**. Please check this help page to know more about the coercion methods for **FLR** objects.

More information

For further explanations on the use of the two plotting platforms available for **FLR** classes, please refer to the **ggplotFL** plotting **FLR** objects with **ggplot2** and Plotting **FLR** objects using **lattice** tutorials.

Running a stock assessment

An important step in providing management advice for a fish stock is the estimation of current status, past history and stock productivity through a stock assessment (SA) model fit. The *FLStock* object we have created contains some of the inputs required for most SA models: catches, natural mortality, and maturity. What is now needed is some indication of changes in abundance over the period of exploitation, an index of abundance, derived from either research surveys or catch-per-unit-effort of commercial fleets.

Index of abundance

FLR has specific classes for this information, **FLIndex** or when multiple indices are available **FLIndices**. An index of abundance for the North Sea plaice stock is available as an example dataset in *FLCore*, and can be

loaded using the *data* command

```
data(ple4.index)
```

This time series of abundances at age, covering from 1996 to 2017, was obtained during the ICES International Bottom Trawl Survey, as is one of the indices used by the Working Group on the Assessment of Demersal Stocks in the North Sea and Skagerrak (WGNSSK).

The *FLIndex* class is able to contain not only the index of abundance, in this case as an age-structured *FLQuant*, but also other information on variance, associated catches, effort, selectivity and catchability, which might be used by certain methods.

```
summary(ple4.index)
```

An object of class "FLIndex"

Name: BTS-Combined (all)

Description: Plaice in IV

. Imported from VPA file.

Type : numbers

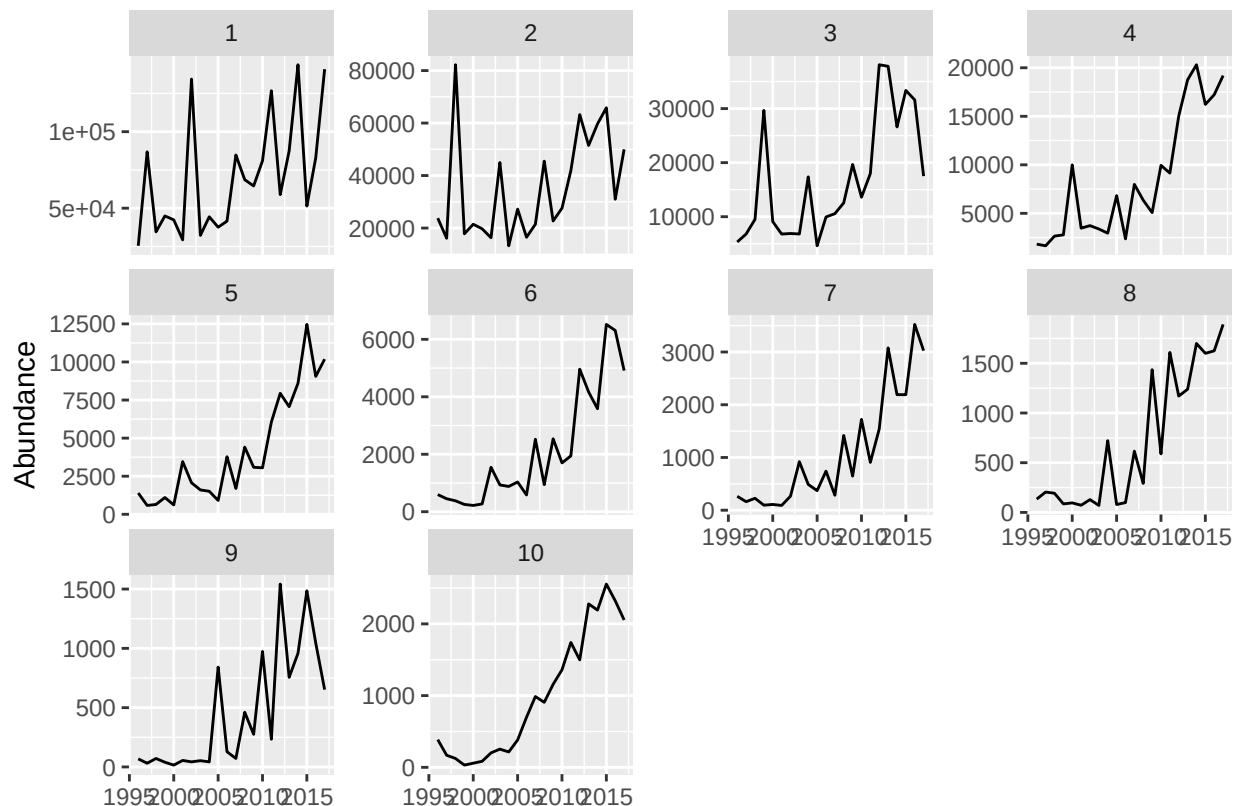
Distribution :

Quant: age

Dims:	age	year	unit	season	area	iter
	10	22	1	1	1	1

Range:	min	max	pgroup	minyear	maxyear	startf	endf
	1	10	1	1996	2017	0.6453376	0.6453376

```
plot(ple4.index)
```



The *range* slot in this class contains information about the time period over which the survey takes place, as proportions of the year, in this case from the beginning of August to the end of September.

```
range(ple4.index)[c("startf", "endf")]
```

```
      startf      endf  
0.6453376 0.6453376
```

a4a statistical catch-at-age

The FLa4a package provides an **FLR** implementation of the *a4a* model, a statistical catch-at-age model of medium complexity that has been designed to make use of the power and flexibility of R's formula-based model notation (as used for example in `lm`), while giving access to a solid and fast minimization algorithm implemented using AD Model Builder.

```
library(FLa4a)
```

A first go at fitting this model to the *ple4* and *ple4.index* datasets requires a single line of code

```
fit <- sca(ple4, ple4.index)
```

This returns an object of class `a4aFitSA` that contains the results of the model fit, including the estimated catch-at-age, and the derived abundances and fishing mortality at age

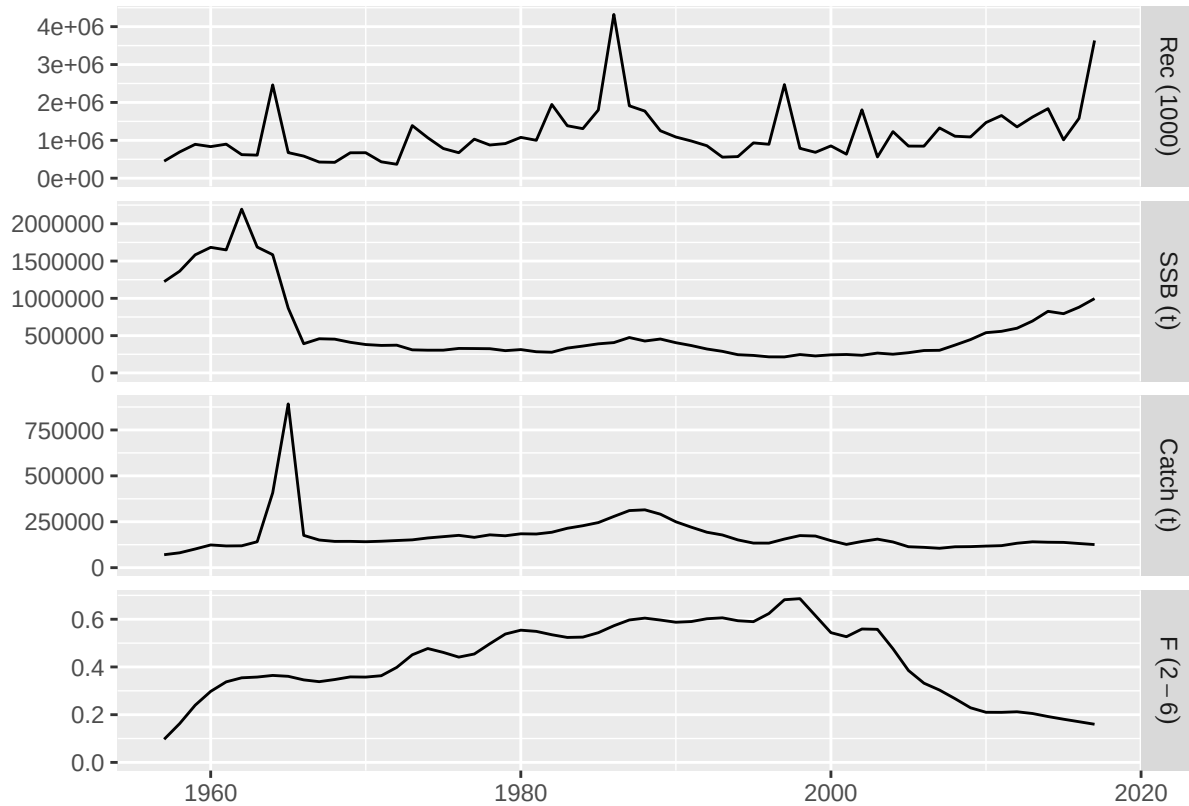
```
summary(fit)
```

An object of class "a4aFitSA"

```
Name: PLE  
Description: Plaice in IV. ICES WGNSSK 2018. FLAAP  
Quant: age  
Dims:  age  year   unit   season  area   iter  
      10  61   1    1    1    1  
  
Range:  min max pgroup  minyear  maxyear  minfbar  maxfbar  
       1  10  10  1957    2017     2     6
```

which we can use to update the *FLStock* object

```
stk <- ple4 + fit  
plot(stk)
```



Among other methods, *FLa4a* provides a range of tools for inspecting the quality of the model fit.

Exploring the stock-recruitment relationship

Fitting a model for the relationship between stock abundance and expected recruitment is necessary for forecasting the future dynamics of the stock under various levels of fishing mortality, the central element of fisheries management advice. Certain stock assessment models, for example most statistical catch-at-age models, include the stock-recruitment model in the estimated parameters, but for others, like Virtual Population Analysis, this is a setup carried out once the stock and population abundance have been obtained.

Both inputs and outputs for stock-recruitment models are stored in an object of the `FLSR` class which we can create directly from the stock assessment results by coercion using

```
plsr <- as.FLSR(stk)
```

The resulting `FLSR` object contains the series of recruitment `rec`, the abundance of the first age, and the stock reproductive potential, in this case the spawning stock biomass `ssb`. Both series are already displaced by the age of recruitment, 1 in this case.

```
summary(plsr)
```

An object of class "FLSR"

Name: PLE

Description: 'rec' and 'ssb' slots obtained from a 'FLStock' object

Quant: age

Dims:	age	year	unit	season	area	iter
1	60	1	1	1	1	1

Range: min minyear max maxyear

```
1 1958 1 2017
```

```
Model: list()
An object of class "FLPar"
character(0)
units: NA
Log-likelihood: NA(NA)
Variance-covariance: <0 x 0 matrix>
```

We now need to choose a model to fit, and *FLCore* provides a number of them already defined in terms of:

- the model formula, kept in the `model` slot
- a function returning the log-likelihood, to be used for minimization, in the `logl` slot
- a function to obtain initial values for the minimization procedure, in the `initial` slot

See the SR Models help page for further details of the available models.

We can now assign the output of the chosen model function, in this case `ricker()`, to the existing object using

```
model(plsr) <- ricker()
```

The model object is ready to fit, through Maximum Likelihood Estimation (MLE), using the `fmle` method defined in *FLCore*. This makes use of R's `optim`, but defaults to the Melder-Mead algorithm.

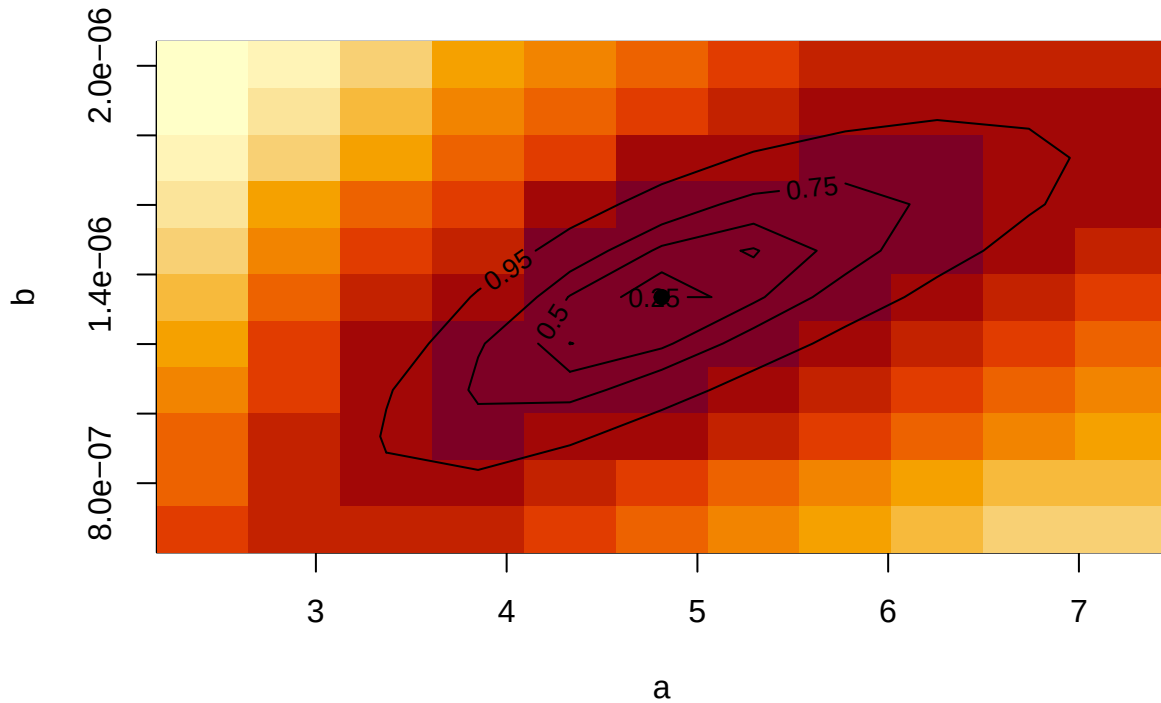
```
plsr <- fmle(plsr)
```

Another look at the object will show us that a number of slots now contain the results of the model fit, among them:

- `fitted` for the estimated recruitment, as an *FLQuant*
- `residuals` for the residuals in log space, *FLQuant*
- `params` for the estimated parameters, as an object of class *FLPar*
- `vcov` for the variance-covariance matrix, as an *array*

The standard plot for the class shows the model fit and a series of useful diagnostics that will help us evaluating its quality and usefulness for future projections. This can be further explored by likelihood profiling over a range of values of the estimated parameters by simply calling the `profile` method of the object

```
profile(plsr)
```



Finally, the model can be used to predict future recruitments if provided with a set of new values of SSB, as in

```
predict(plsr, ssb=FLQuant(rnorm(10, 25e4, sd(ssb(plsr))), dimnames=list(age=1, year=2008:2017)))
```

An object of class "FLQuant"

```
, , unit = unique, season = all, area = unique
```

	year						
age	2008	2009	2010	2011	2012	2013	2014
1	-10008692	1322964	45880	957730	893855	872758	-1209982

	year		
age	2015	2016	2017
1	805894	-1329512	1032422

units: 1000

More information

Please see the Modelling stock recruitment with FLSR tutorial for more examples and information on the stock-recruitment model fitting capabilities of **FLR**.

Estimating reference points

To assess current and future status of a stock we need reference points which we hold to represent desirable states for the stock, usually either in terms of stock abundance, like B_{MSY} , the biomass expected to produce, on average and under the current exploitation pattern, the Maximum Sustainable Yield (MSY), or of fishing mortality, such as the fishing mortality expected to produce MSY , F_{MSY} .

We can use the FLBRP package to calculate a large set of reference points, once we have an estimate of stock status, our *FLStock* object, and of the stock-recruitment relationship, *FLSR*. We can use them both to create the necessary object, of class *FLBRP*, after loading the package

```
library(FLBRP)
plrp <- FLBRP(stk, sr=plsr)
summary(plrp)
```

An object of class "FLBRP"

Name:

Description:

Quant: age

Dims: age year unit season area iter
10 101 1 1 1 1

Range: min max pgroup minfbar maxfbar
1 10 10 2 6

Model: $rec \sim a * ssb * \exp(-b * ssb)$

params
iter a b
1 4.81 1.33e-06

refpts: NA

An object of the *FLBRP* class contains a series of times series of observations on dynamics of both stock and fishery, as well as average quantities computed along a specified time period. A series of slots are designed to contain economic variables, such as **price**, variable costs (**vcost**) and fixed costs (**fcost**), which will need to be provided. When those data are available, economic reference points can also be calculated. Finally, a vector of fishing mortality values, **fbar**, has been created, by default to be of length 101 and with values going from $F = 0$ to $F = 4$, to be used in the calculation procedure.

We can now proceed with the calculation of reference points, through the **brp** method

```
plrp <- brp(plrp)
```

and the extraction of the values obtained, stored in the **refpts** slot of the object

```
refpts(plrp)
```

An object of class "FLPar"

	quant						
refpt	harvest	yield	rec	ssb	biomass	revenue	cost
virgin	0.00e+00	0.00e+00	6.14e+05	2.10e+06	2.16e+06	NA	NA
msy	2.63e-01	1.03e+05	1.31e+06	8.89e+05	1.01e+06	NA	NA
crash	5.10e-01	9.18e-07	1.71e-05	3.56e-06	4.87e-06	NA	NA
f0.1	1.52e-01	8.28e+04	1.06e+06	1.37e+06	1.47e+06	NA	NA
fmax	2.05e-01	9.73e+04	1.20e+06	1.13e+06	1.25e+06	NA	NA
spr.30	1.91e-01	9.43e+04	1.17e+06	1.20e+06	1.31e+06	NA	NA
mey	NA	NA	NA	NA	NA	NA	NA

	quant
refpt	profit
virgin	NA
msy	NA
crash	NA
f0.1	NA
fmax	NA
spr.30	NA

```

    mey      NA
units:  NA

```

This slot contains the estimates for a series of reference points and for a number of quantities. For example, the value stored at the intersection of `msy` and `harvest` corresponds to F_{MSY} , while that of `virgin` and `ssb` is the estimate of virgin stock biomass. We can, for example, extract the two standard MSY-related reference points as a named vector using

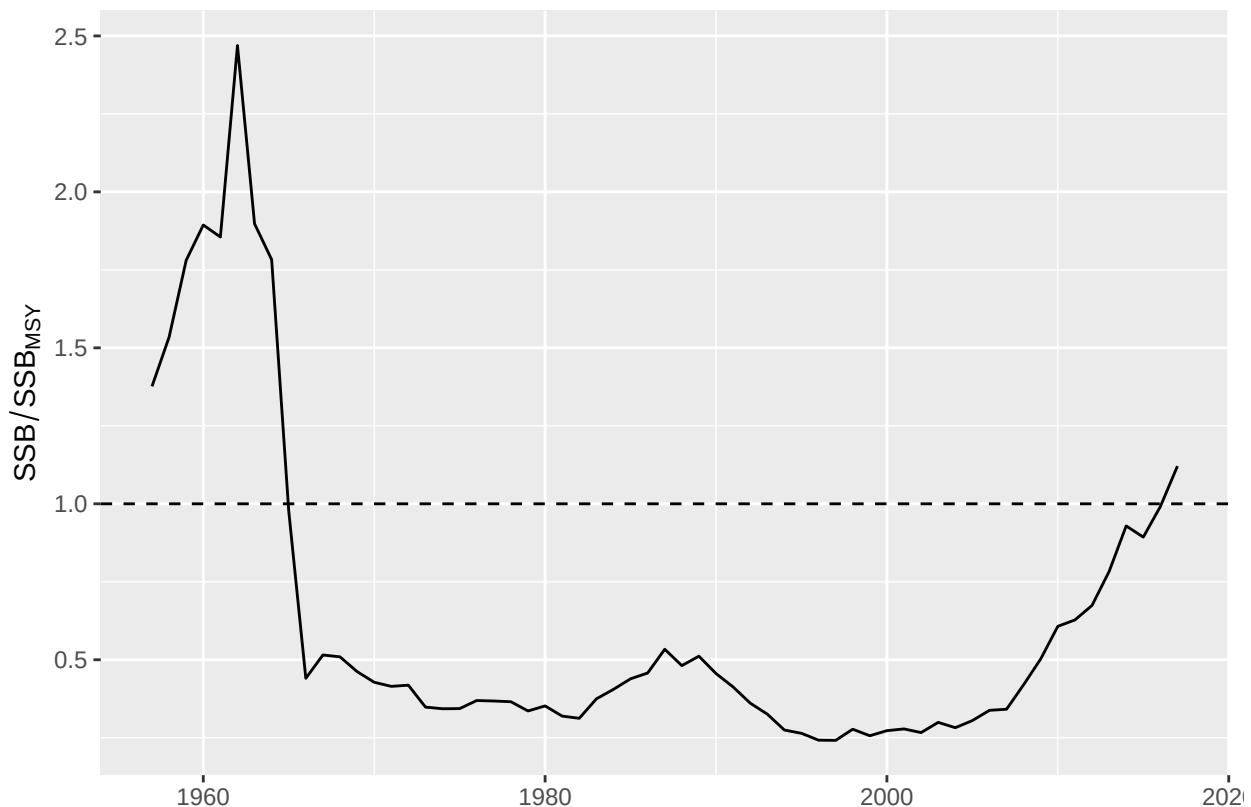
```
pmsy <- refpts(plrp)["msy", c("harvest", "ssb"), drop=TRUE]
```

to then use them to visualize the status of the stock in relation to these points, for example

```

plot(ssb(stk) / pmsy["ssb"]) + geom_hline(aes(yintercept=1), linetype=2) +
  ylab(expression(SSB / SSBMSY))

```



More information

For more information on *FLBRP*, please see the Reference points for fisheries management with FLBRP tutorial.

Forecasting under different scenarios

Our final step in this quick tour takes us to the tools in *FLR* for forecasting the future responses of the stock to alternative management decisions. This example concerns a Short Term Forecast, in which the effect of the current management, for example a maximum catch limit or TAC, on the present year is forecast, and then a series of alternative increases or decreases in catch are applied to the object representing our current understanding of the stock status and dynamics.

The FFlasher package contains the current implementation of the forecasting mechanism in *FLR*.

```
library(FLasher)
```

The key element in the *FLasher* interface is the *fwdControl* class, which allows us to specify the targets and limits that we want the stock to achieve or remain within. But first we need to make some assumptions about the future values of certain quantities, both on the biology (natural mortality, maturity) and the fishery (selectivity, proportion of discards). A common approach for this kind of forecast is to use averages over the last few years (for example three, the default). This can be done simply in *FLR* using the *fwdWindow* method in the *FLCore* package.

```
proj <- fwdWindow(stk, end=2020)
```

Our new object extends up to 2020, and the values for certain stocks have been extended using the mean of the last three years, for example for the weight-at-age in the stock

```
stock.wt(proj)[, ac(2016:2020)]
```

An object of class "FLQuant"

```
, , unit = unique, season = all, area = unique
```

	year				
age	2016	2017	2018	2019	2020
1	0.0300	0.0320	0.0287	0.0287	0.0287
2	0.0660	0.0690	0.0667	0.0667	0.0667
3	0.1170	0.1320	0.1230	0.1230	0.1230
4	0.1980	0.1810	0.1953	0.1953	0.1953
5	0.2600	0.2700	0.2697	0.2697	0.2697
6	0.3290	0.3330	0.3283	0.3283	0.3283
7	0.3800	0.3590	0.3727	0.3727	0.3727
8	0.4340	0.4580	0.4423	0.4423	0.4423
9	0.4790	0.4760	0.4733	0.4733	0.4733
10	0.5140	0.5570	0.5093	0.5093	0.5093

units: kg

We can now construct a *fwdControl* object that projects the stock as if catches in the current year, 2018 in this case, were those of the agreed TAC, 85,000 t, and F was then kept in 2019 and 2020 at the same levels as in 2017. The main information to be passed to *fwdControl* object is

- *year*
- *quant*, the quantity to be projected, for example *ssb*, *f* or *catch*
- *value*, the actual value to be obtained

```
TAC <- 85000
```

```
Flevel <- fbar(stk)[,"2017"]
```

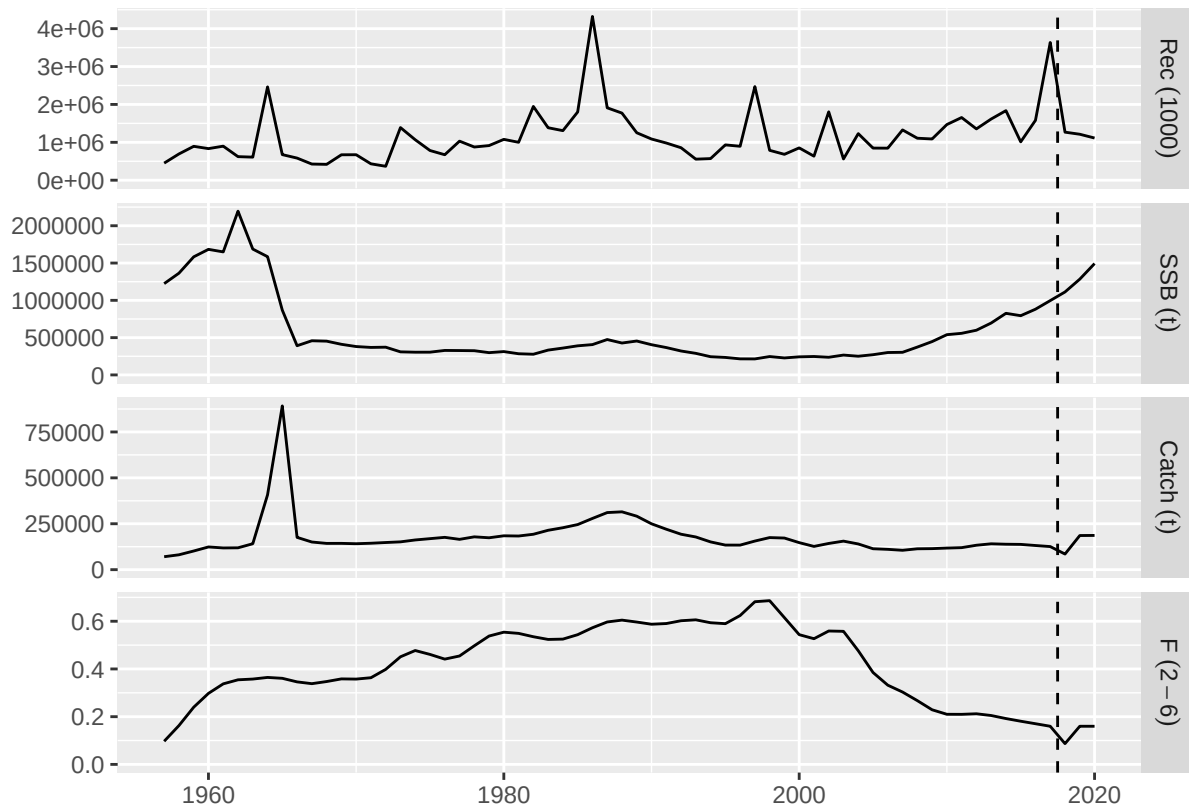
```
ctrl <- fwdControl(year=2018:2020, quant=c("catch", "f", "f"), value=c(TAC, Flevel, Flevel))
```

Now the *fwd* method will carry out the projection using those three object: stock, stock-recruitment and control

```
proj <- fwd(proj, control=ctrl, sr=plsr)
```

and the results can be visualized

```
plot(proj) + geom_vline(aes(xintercept=2017.5), linetype=2)
```



More information

Please see the tutorial *Forecasting on the Medium Term* for advice using *FLasher* for more examples on how to use the capabilities of *FLasher* to forecast population dynamics under various scenarios.

More on FLR

Please visit our website for more information on the **FLR** packages, tutorials and examples of use, and instructions for getting in touch if you want to report a bug, ask a question or get involved in the development of the **FLR** toolset. The project is run by a team of fisheries scientists that decided to share the R code they were writing and realised that using a common platform for Quantitative Fisheries Science in R could break the Babel tower spell that J. Schnute first identified

“The cosmic plan for confounding software languages seems to be working remarkably well among the community of quantitative fishery scientists!”

Schnute et al. 2007

References

L. T. Kell, I. Mosqueira, P. Grosjean, J-M. Fromentin, D. Garcia, R. Hillary, E. Jardim, S. Mardle, M. A. Pastoors, J. J. Poos, F. Scott, R. D. Scott; FLR: an open-source framework for the evaluation and development of management strategies. *ICES J Mar Sci* 2007; 64 (4): 640-646. doi: 10.1093/icesjms/fsm012.

More information

- You can submit bug reports, questions or suggestions on this tutorial at <https://github.com/flr/doc/issues>.

- Alternatively, send a pull request to <https://github.com/flr/doc/>.
- For more information on the FLR Project for Quantitative Fisheries Science in R, visit the FLR webpage: <http://flrproject.org>.

Software Versions

- R version 4.5.2 (2025-10-31)
- FLCore: 2.6.23
- ggplotFL: 2.7.1
- FLa4a: 1.9.5
- FLBRP: 2.5.9.9051
- FLasher: 0.7.2
- **Compiled:** Fri Feb 20 11:07:26 2026

License

This document is licensed under the Creative Commons Attribution-ShareAlike 4.0 International license.

Author information

Iago MOSQUEIRA. Wageningen Marine Research (WMR), IJmuiden, The Netherlands.

Ernesto Jardim. Instituto Portugues do Mar e Atmosfera (IPMA), Lisbon, PortugalNational Institute of Aquatic Resources, Technical University of Denmark (DTU-Aqua), Copenhagen, Denmark.